



# Qdt Integration's Manual for FC7xxx Series

*Rev: 1.2.0*

Target Devices:

FC7300F8MDQ, FC7300F8MDT

FC7300F4MDD, FC7300F4MDS

FC7240F2MDS

## Table of Contents

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>Chapter 1. Introduction .....</b>                    | <b>2</b>  |
| 1.1. Introduction .....                                 | 2         |
| <b>Chapter 2. Building .....</b>                        | <b>3</b>  |
| 2.1. Dependencies on Other Modules .....                | 3         |
| 2.2. Files Required for Compile .....                   | 3         |
| 2.3. Add Plug-ins .....                                 | 4         |
| <b>Chapter 3. Memory .....</b>                          | <b>5</b>  |
| 3.1. Sections in Memory Map .....                       | 5         |
| <b>Chapter 4. Exclusive Area .....</b>                  | <b>6</b>  |
| <b>Chapter 5. Interrupt Service Routine (ISR) .....</b> | <b>7</b>  |
| <b>Chapter 6. Error Report .....</b>                    | <b>8</b>  |
| 6.1. Det .....                                          | 8         |
| 6.2. Dem .....                                          | 9         |
| <b>Chapter 7. Function Calls .....</b>                  | <b>10</b> |
| 7.1. Function Calls during Startup .....                | 10        |
| 7.2. Function Calls during Shutdown .....               | 10        |
| 7.3. Function Calls during Wake-up .....                | 10        |
| 7.4. Function Calls during Runtime .....                | 10        |
| <b>Chapter 8. Other Requirements .....</b>              | <b>11</b> |
| 8.1. Notification, Callback, Callout .....              | 11        |
| 8.2. Macros .....                                       | 11        |
| <b>Chapter 9. Integration Steps .....</b>               | <b>12</b> |

## **Chapter 1. Introduction**

### **1.1. Introduction**

This integration manual describes the integration requirements for the QDT module.

## Chapter 2. Building

### 2.1. Dependencies on Other Modules

Module configuration dependency

- Mcu: This module provides the clock reference point for QDT module
- Port: This module provides the port configurations if want output QDT signal through pin.
- Common: This module is the basic module which used to choose the chip.
- OS: This is used for QDT main function reference.
- EcuM: is required for selecting the reference to the wakeup source for every QDT controller.
- Det: is required for signaling the default error detection (parameters out of range, null pointers, etc)

Module build dependency

- Common: This module provides common type for other modules.
- Rte: This module provides APIs to protect/unprotect some parts of code from interrupts (Exclusive Areas).
- Det: This module is necessary for tracing Development errors.
- EcuM: This module provides the EcuM callback functions when wakeup support is enabled.

Module initialization dependency

- Mcu: To use the QDT module, the user needs to initialize the MCU module first.
- Port: To output QDT signal, the user needs to initialize PORT module first and assign the UART pins.

### 2.2. Files Required for Compile

Qdt module files:

- \_MCAL/Src/Qdt/Src/CDD\_Qdt.c
- \_MCAL/Src/Qdt/Src/Qdt\_Hal.c
- \_MCAL/Src/Qdt/Src/Qdt\_HWA.c
- \_MCAL/Src/Qdt/Src/Qdt\_Isr.c
- \_MCAL/Src/Qdt/include/CDD\_Qdt.h
- \_MCAL/Src/Qdt/include/Qdt\_Hal.h
- \_MCAL/Src/Qdt/include/Qdt\_HWA.h
- \_MCAL/Src/Qdt/include/Qdt\_MemMap.h
- \_MCAL/Src/Qdt/include/Qdt\_Reg.h
- \_MCAL\_generate/src/CDD\_Qdt\_cfg.c
- \_MCAL\_generate/include CDD\_Qdt\_cfg.h

Common module files:

- \_MCAL/Src/Common/include/Mcal.h

- `_MCAL/Src/Common/include/Std_Types.h`
- `_MCAL/Src/Common/include/Platform_Types.h`
- `_MCAL/Src/Common/include/Compiler.h`
- `_MCAL/Src/Common/include/Compiler_Cfg.h`
- `_MCAL/Src/Common/include/CompilerDefinition.h`

Det module files:

- `Det.h`

Rte module files:

- `SchM_Qdt.h`

### 2.3. Add Plug-ins

QDT module plug-ins are developed for EB tresos Studio, so, to use QDT plug-ins on the EB tresos Studio, the user needs to add the QDT module to the EB plug-ins folder first.

1. Copy the QDT module (`_MCAL/EB_Plugins/eclipse/plugins/QDT`) folder to EB tresos plug-ins (`EB/tresos/plugins/`) folder.
2. Set the QDT module output location folder for the generated source file and header files.
3. Use EB tresos to configure the QDT module and generate source and header files.

## Chapter 3. Memory

### 3.1. Sections in Memory Map

| Section Name                                                                                                                                                                                                                                                                                                        | Section Type       | Description                                                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| QDT_START_SEC_CONFIG_DATA_8<br>QDT_STOP_SEC_CONFIG_DATA_8<br>QDT_START_SEC_CONFIG_DATA_16<br>QDT_STOP_SEC_CONFIG_DATA_16<br>QDT_START_SEC_CONFIG_DATA_32<br>QDT_STOP_SEC_CONFIG_DATA_32                                                                                                                             | Variables          | These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code(data).                         |
| QDT_START_SEC_CONFIG_DATA_UNSPECIFIED<br>QDT_STOP_SEC_CONFIG_DATA_UNSPECIFIED                                                                                                                                                                                                                                       | Configuration Data | Start and stop of Memory Section for Config Data.                                                                                                                     |
| QDT_START_SEC_CONST_BOOLEAN<br>QDT_STOP_SEC_CONST_BOOLEAN<br>QDT_START_SEC_CONST_8<br>QDT_STOP_SEC_CONST_8<br>QDT_START_SEC_CONST_16<br>QDT_STOP_SEC_CONST_16<br>QDT_START_SEC_CONST_32<br>QDT_STOP_SEC_CONST_32<br>QDT_START_SEC_CONST_UNSPECIFIED<br>QDT_STOP_SEC_CONST_UNSPECIFIED                               | Variables          | These are all the sections used for variables which have to be aligned to 8/16/32 bit or boolean. These variables are read only (rodata).                             |
| QDT_START_SEC_CODE<br>QDT_STOP_SEC_CODE<br>QDT_START_SEC_RAMCODE<br>QDT_STOP_SEC_RAMCODE<br>QDT_START_SEC_CODE_AC<br>QDT_STOP_SEC_CODE_AC                                                                                                                                                                           | Code               | Start and stop of memory Section for Code (text).                                                                                                                     |
| QDT_START_SEC_CODE<br>QDT_STOP_SEC_CODE<br>QDT_START_SEC_RAMCODE<br>QDT_STOP_SEC_RAMCODE<br>QDT_START_SEC_CODE_AC<br>QDT_STOP_SEC_CODE_AC                                                                                                                                                                           | Variables          | These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are never cleared and never initialized by startup code (bss). |
| QDT_START_SEC_VAR_INIT_BOOLEAN<br>QDT_STOP_SEC_VAR_INIT_BOOLEAN<br>QDT_START_SEC_VAR_INIT_8<br>QDT_STOP_SEC_VAR_INIT_8<br>QDT_START_SEC_VAR_INIT_16<br>QDT_STOP_SEC_VAR_INIT_16<br>QDT_START_SEC_VAR_INIT_32<br>QDT_STOP_SEC_VAR_INIT_32<br>QDT_START_SEC_VAR_INIT_UNSPECIFIED<br>QDT_STOP_SEC_VAR_INIT_UNSPECIFIED | Variables          | These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code (data).                        |

## Chapter 4. Exclusive Area

QDT module using the services of Schedule Manger (SchM) for entering and exiting critical regions. The following critical regions are used in the QDT driver: • QDT\_Hal.c:

- SchM\_Enter\_Qdt\_QDT\_EXCLUSIVE\_AREA\_00 is used in Qdt\_HL\_Instance\_Init.
- SchM\_Enter\_Qdt\_QDT\_EXCLUSIVE\_AREA\_01 is used in Qdt\_ML\_Channel\_Init.
- SchM\_Enter\_Qdt\_QDT\_EXCLUSIVE\_AREA\_02 is used in. Qdt\_ML\_Channel\_Init
- SchM\_Enter\_Qdt\_QDT\_EXCLUSIVE\_AREA\_03 is used in Qdt\_ML\_Channel\_Init.
- SchM\_Enter\_Qdt\_QDT\_EXCLUSIVE\_AREA\_04 is used in Qdt\_ML\_Channel\_Init.
- SchM\_Enter\_Qdt\_QDT\_EXCLUSIVE\_AREA\_05 is used in Qdt\_ML\_Channel\_Init.
- SchM\_Enter\_Qdt\_QDT\_EXCLUSIVE\_AREA\_06 is used in Qdt\_ML\_Channel\_Init.
- SchM\_Enter\_Qdt\_QDT\_EXCLUSIVE\_AREA\_07 is used in Qdt\_ML\_Channel\_Init.
- SchM\_Enter\_Qdt\_QDT\_EXCLUSIVE\_AREA\_08 is used in Qdt\_ML\_Channel\_Init.
- SchM\_Enter\_Qdt\_QDT\_EXCLUSIVE\_AREA\_09 is used in Qdt\_ML\_Channel\_Init.
- SchM\_Enter\_Qdt\_QDT\_EXCLUSIVE\_AREA\_10 is used in Qdt\_ML\_Channel\_Init.
- SchM\_Enter\_Qdt\_QDT\_EXCLUSIVE\_AREA\_11 is used in Qdt\_HL\_Instance\_DeInit.
- SchM\_Enter\_Qdt\_QDT\_EXCLUSIVE\_AREA\_12 is used in Qdt\_LL\_Channel\_DeInit.
- SchM\_Enter\_Qdt\_QDT\_EXCLUSIVE\_AREA\_13 is used in QDT\_Hal\_ReStartMeasurement.
- SchM\_Enter\_Qdt\_QDT\_EXCLUSIVE\_AREA\_15 is used in QDT\_Hal\_ClearChannelFlag.

# Chapter 5. Interrupt Service Routine (ISR)

| Instance | Interrupt Name  | IRQ Number (NVIC Interrupt ID) |
|----------|-----------------|--------------------------------|
| QDT0     | QDT_IsrQDT0_ALL | 191                            |
| QDT1     | QDT_IsrQDT1_ALL | 192                            |
| QDT2     | QDT_IsrQDT2_ALL | 193                            |
| QDT3     | QDT_IsrQDT3_ALL | 194                            |



## Chapter 6. Error Report

### 6.1. Det

| Function Name          | Error Type                                                                               |
|------------------------|------------------------------------------------------------------------------------------|
| QDT_Init               | QDT_INITIALIZED;<br>QDT_E_INIT_FAILED_U8;<br>QDT_E_PARTITION_MAPPING;<br>QDT_E_PARAM_U8; |
| Sent_GetVersionInfo    | QDT_E_PARAM_U8;                                                                          |
| QDT_DeInit             | QDT_E_UNINIT_U8;<br>QDT_E_PARTITION_MAPPING;                                             |
| QDT_ReStartMeasurement | QDT_E_INVALID_CHANNEL_U8;<br>QDT_E_PARAM_U8;<br>QDT_E_UNINIT_U8;                         |
| QDT_GetChannelFlag     | QDT_E_INVALID_CHANNEL_U8;<br>QDT_E_UNINIT_U8;                                            |
| QDT_ClearChannelFlag   | QDT_E_INVALID_CHANNEL_U8;<br>QDT_E_UNINIT_U8;                                            |
| QDT_GetEdgeNumber      | QDT_E_PARAM_U8;<br>QDT_E_UNINIT_U8;                                                      |
| QDT_GetCV              | QDT_E_PARAM_U8;<br>QDT_E_UNINIT_U8;                                                      |
| QDT_GetREVCNT          | QDT_E_PARAM_U8;<br>QDT_E_UNINIT_U8;                                                      |
| QDT_GetPOSCNT          | QDT_E_PARAM_U8;<br>QDT_E_UNINIT_U8;                                                      |
| QDT_ResetPOSCNT        | QDT_E_UNINIT_U8;                                                                         |
| QDT_GetPOSCNT_HOLD     | QDT_E_PARAM_U8;<br>QDT_E_UNINIT_U8;                                                      |
| QDT_GetPOSDCNT         | QDT_E_PARAM_U8;<br>QDT_E_UNINIT_U8;                                                      |
| QDT_GetPOSDCNT_HOLD    | QDT_E_PARAM_U8;<br>QDT_E_UNINIT_U8;                                                      |
| QDT_GetLECNT           | QDT_E_PARAM_U8;<br>QDT_E_UNINIT_U8;                                                      |

| Function Name          | Error Type                          |
|------------------------|-------------------------------------|
| QDT_GetLECNT_HOLD      | QDT_E_PARAM_U8;<br>QDT_E_UNINIT_U8; |
| QDT_GetPOSDTMRCNT      | QDT_E_PARAM_U8;<br>QDT_E_UNINIT_U8; |
| QDT_GetPOSDTMRCNT_HOLD | QDT_E_PARAM_U8;<br>QDT_E_UNINIT_U8; |
| QDT_GetSpeed           | QDT_E_PARAM_U8;<br>QDT_E_UNINIT_U8; |

## 6.2. Dem

None

## Chapter 7. Function Calls

### 7.1. Function Calls during Startup

QDT shall be initialized during STARTUP phase of EcuM initialization. The API to be called for this is QDT\_Init(). The MCU module should be initialized before the QDT is initialized. The QDT module shall be initialized by QDT\_Init(<&QDT\_Configuration>) service call during the start-up before the QDT peripherals are used. Please note that GPIO pins used for connection of QDT physical layer have to be properly assigned to the desired module prior to the QDT initialization: so the MCU and PORT modules shall be initialized before QDT is initialized. After the QDT module is initialized, each QDT controller has to be initialized as well before using it. This is also done by the QDT\_Init(<&QDT\_Configuration>) service.

### 7.2. Function Calls during Shutdown

None

### 7.3. Function Calls during Wake-up

None

### 7.4. Function Calls during Runtime

None

## Chapter 8. Other Requirements

### 8.1. Notification, Callback, Callout

- Notification

There are no callback functions within the QDT driver.

- Callback

There are no callback functions within the QDT driver.

- Callout

These functions are optional: TimerOverflowNotification, WdogNotification and ChannelNotification.

### 8.2. Macros

Please check various definitions available in Common module's include file Mcal.h for details.

- If OS is not used:

AUTOSAR\_OS\_NOT\_USED needs to be defined.

- If OS is used:

Do not define AUTOSAR\_OS\_NOT\_USED.

## Chapter 9. Integration Steps

- 1) Configure the QDT module and generate configuration files (please refer to Building chapter for details).
- 2) Configure appropriate memory sections in linker file or other (please refer to Memory chapter for details).
- 3) Map interrupt notification to their vector locations (please refer to chapter ISR for details).
- 4) Build the QDT module with dependent modules.